

LAPORAN PRAKTIKUM
MACHINE LEARNING (PRAKTIKUM)



Nama : Shendy Tria Amelyana
NIM 434231003
Kelas : C1

DOSEN PENGAMPU :
INDAH WERDININGSIH, S.SI., M.Kom.

PROGRAM STUDI DIPLOMA IV TEKNIK INFORMATIKA
FAKULTAS VOKASI
UNIVERSITAS AIRLANGGA
SURABAYA
2025

Pendahuluan

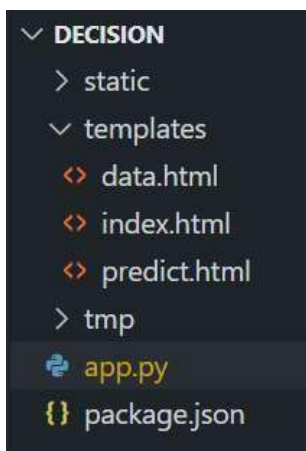
Praktikum ini bertujuan untuk mengembangkan aplikasi web menggunakan Flask yang mengintegrasikan model *Machine Learning* berbasis *Decision Tree Classifier*. Aplikasi ini memungkinkan pengguna untuk mengunggah dataset dalam format CSV atau XLSX, melatih model klasifikasi, menampilkan visualisasi pohon keputusan, dan melakukan prediksi berdasarkan input pengguna. Fokus utama laporan ini adalah pada penjelasan cuplikan kode dari `app.py`, yang mencakup preprocessing data, pelatihan model, dan integrasi dengan antarmuka web. Route `/static` dan `/tmp` juga didukung untuk file statis dan sementara.

Tujuan

1. Memahami implementasi kode Flask dengan integrasi *Decision Tree Classifier*.
2. Menjelaskan cuplikan kode kunci untuk preprocessing, pelatihan, visualisasi, dan prediksi.
3. Mengintegrasikan model ML dengan web app sambil menangani file statis dan sementara.

Alat dan Bahan

- Software: Python 3.x, Flask, scikit-learn, pandas, Graphviz, numpy.
- File:



Langkah Kerja

1. Install library: `pip install flask pandas scikit-learn graphviz numpy openpyxl`.
2. Siapkan struktur proyek dengan `app.py` dan template HTML.
3. Jalankan python `app.py` dan uji melalui browser.

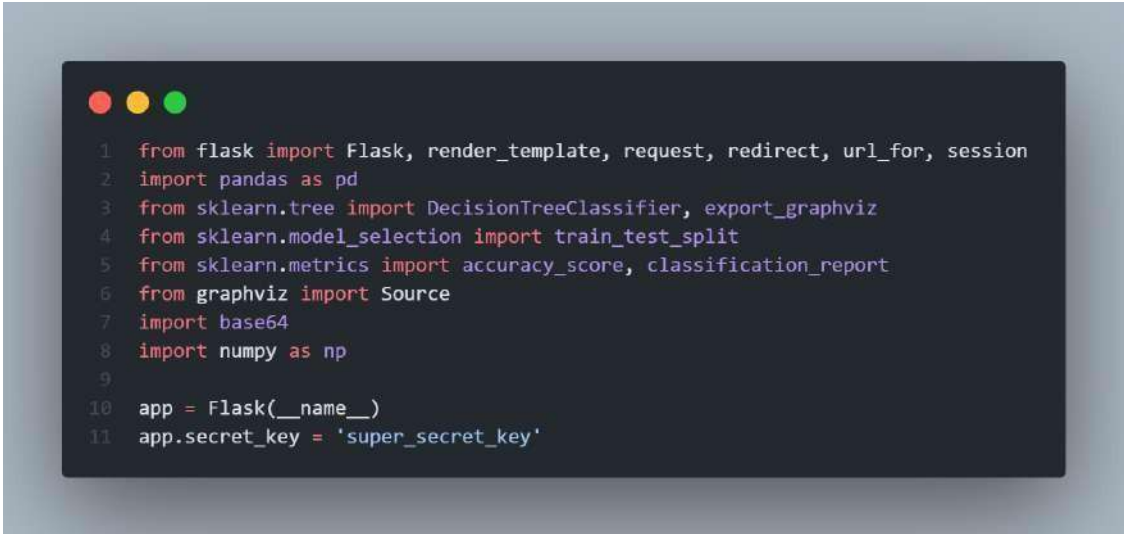
Kode yang Digunakan

Bagian ini difokuskan pada penjelasan cuplikan kode dari `app.py`. Kode lengkap menggunakan Flask untuk routing, pandas untuk data handling, scikit-learn untuk model

Decision Tree, dan Graphviz untuk visualisasi. Variabel global digunakan untuk menyimpan state (model, fitur, dll.), meskipun ini bukan praktik terbaik untuk produksi. Penjelasan disertai cuplikan kode kunci untuk kejelasan.

1. Import Library dan Inisialisasi App

Cuplikan kode ini mengimpor library yang diperlukan dan menginisialisasi aplikasi Flask dengan secret key untuk session. Library seperti pandas untuk membaca data, sklearn untuk model dan metrik, serta graphviz dan base64 untuk visualisasi pohon keputusan.



```
1 from flask import Flask, render_template, request, redirect, url_for, session
2 import pandas as pd
3 from sklearn.tree import DecisionTreeClassifier, export_graphviz
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, classification_report
6 from graphviz import Source
7 import base64
8 import numpy as np
9
10 app = Flask(__name__)
11 app.secret_key = 'super_secret_key'
```

Penjelasan:

- Flask digunakan untuk membuat web server dengan route seperti /upload dan /predict.
- pandas (pd) untuk membaca dan memanipulasi data dari file CSV/XLSX.
- sklearn.tree untuk *DecisionTreeClassifier* dan ekspor pohon ke format DOT (untuk Graphviz).
- sklearn.model_selection untuk membagi data menjadi train/test set.
- sklearn.metrics untuk menghitung akurasi dan laporan klasifikasi.
- graphviz.Source untuk merender pohon keputusan menjadi gambar PNG.
- base64 untuk mengonversi gambar PNG menjadi string base64 agar bisa ditampilkan langsung di HTML tanpa menyimpan file fisik.
- Variabel global seperti global_model dideklarasikan untuk menyimpan state aplikasi (model, fitur, dll.) di seluruh request.

2. Route Utama (/): Halaman Index

Cuplikan ini mendefinisikan route utama yang merender template index.html untuk form upload file.

```

1 @app.route('/')
2 def index():
3     return render_template('index.html')

```

Penjelasan:

- Route ini adalah entry point aplikasi, menampilkan halaman sederhana dengan form untuk mengunggah file dataset.
- `render_template` memanggil file HTML dari folder templates, yang berisi form dengan `enctype="multipart/form-data"` untuk upload file.

3. Route /upload: Proses Upload dan Pelatihan Model

Ini adalah bagian inti di mana file diunggah, data diproses, model dilatih, dan hasil dievaluasi.

Cuplikan ini menangani pembacaan file, preprocessing, pelatihan, visualisasi, dan evaluasi.

```

1 @app.route('/upload', methods=['POST'])
2 def upload_file():
3     global global_model, global_feature_names, global_target_names, global_df_processed
4     global global_raw_df, global_all_data_table, global_grouped_data
5     global global_tree_visualization_base64, global_accuracy, global_classification_rep
6     global global_target_column, global_original_columns
7
8     if 'file' not in request.files:
9         return redirect(url_for('index'))
10    file = request.files['file']
11    if file.filename == '':
12        return redirect(url_for('index'))
13
14    try:
15        # baca file (csv / xlsx)
16        if file.filename.endswith('.csv'):
17            df = pd.read_csv(file)
18        elif file.filename.endswith('.xlsx'):
19            df = pd.read_excel(file)
20        else:
21            return "Unsupported file type", 400
22
23        # simpan raw df di memori (DEV only)
24        global_raw_df = df.copy()
25        global_original_columns = df.columns.tolist()
26        global_target_column = df.columns[-1]
27
28        # tampilkan tabel & grouping (tidak disimpan di session)
29        all_data_table = df.head(10).to_html(classes='table table-striped', index=False)
30        grouped_data = df.groupby(global_target_column).head(5).to_html(classes='table table-striped', index=False)
31
32        # preprocessing: one-hot encoding pada fitur (drop first untuk menghindari multikolinearitas)
33        df_processed = pd.get_dummies(df.drop(columns=[global_target_column]), drop_first=True)
34        X = df_processed
35        y = df[global_target_column]
36

```

```

37 # split & train
38 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
39
40 training_testing_summary = {
41     'X_train_shape': X_train.shape,
42     'X_test_shape': X_test.shape,
43     'y_train_shape': y_train.shape,
44     'y_test_shape': y_test.shape
45 }
46
47 # latih decision tree (default, kamu bisa atur max_depth jika mau)
48 dt_classifier = DecisionTreeClassifier(max_depth=4, random_state=42)
49 dt_classifier.fit(X_train, y_train)
50
51 # simpan model & metadata global
52 global_model = dt_classifier
53 global_feature_names = X.columns.tolist()
54 global_target_names = dt_classifier.classes_.tolist()
55 global_df_processed = df_processed
56 global_all_data_table = all_data_table
57 global_grouped_data = grouped_data
58
59 # export tree (graphviz) + base64 (jika available)
60 try:
61     dot_data = export_graphviz(
62         dt_classifier,
63         out_file=None,
64         feature_names=global_feature_names,
65         class_names=[str(c) for c in global_target_names],
66         filled=True, rounded=True, special_characters=True
67     )
68     graph = Source(dot_data, format="png")
69     png_output = graph.pipe(format='png')
70     global_tree_visualization_base64 = base64.b64encode(png_output).decode('utf-8')
71 except Exception as e:
72     # jika graphviz gagal, tidak fatal
73     print("Graphviz export failed:", e)
74     global_tree_visualization_base64 = None
75
76 # evaluasi
77 y_pred = dt_classifier.predict(X_test)
78 global_accuracy = accuracy_score(y_test, y_pred)
79 global_classification_rep = classification_report(y_test, y_pred, output_dict=True)
80
81 # render hasil
82 return render_template(
83     'data.html',
84     table=all_data_table,
85     target_column=global_target_column,
86     columns=global_original_columns,
87     grouped_data=grouped_data,
88     training_testing_summary=training_testing_summary,
89     tree_visualization=global_tree_visualization_base64,
90     accuracy=global_accuracy,
91     classification_report=global_classification_rep,
92     prediction_result=None
93 )
94
95 except Exception as e:
96     return f"Error processing file: {e}", 500

```

Penjelasan:

- Upload File: Mengecek dan membaca file CSV/XLSX menggunakan `pd.read_csv` atau `pd.read_excel`.
- Preprocessing: Menyimpan data raw, mengasumsikan kolom terakhir sebagai target. Menggunakan `pd.get_dummies` dengan `drop_first=True` untuk *one-hot encoding* fitur kategorikal, menghindari *dummy variable trap*.
- Split Data: Membagi data menjadi train/test (70/30) dengan `train_test_split` untuk evaluasi model.

- Pelatihan Model: Membuat *DecisionTreeClassifier* dengan `max_depth=4` untuk membatasi kedalaman pohon dan mencegah *overfitting*. Model dilatih dengan `fit(X_train, y_train)`.
- Visualisasi Pohon: Menggunakan `export_graphviz` untuk menghasilkan DOT data, kemudian dirender ke PNG dengan Source dan di-encode ke base64. Ini memungkinkan tampilan gambar tanpa menyimpan file di /tmp atau static.
- Evaluasi: Memprediksi pada test set dengan `predict`, menghitung akurasi dengan `accuracy_score`, dan laporan klasifikasi dengan `classification_report`.
- Render Template: Mengirim data ke `data.html` untuk ditampilkan, termasuk tabel, summary, visualisasi, dan metrik.

4. Route /predict: Prediksi Berdasarkan Input Pengguna

Cuplikan ini menangani form prediksi (GET) dan proses prediksi (POST), memastikan input selaras dengan encoding training.

```

1 @app.route('/predict', methods=['GET', 'POST'])
2 def predict_input():
3     global global_model, global_feature_names, global_target_column, global_original_columns, global_raw_df
4
5     if global_model is None:
6         return redirect(url_for('index'))
7
8     if request.method == 'POST':
9         # Ambil input user
10        input_data = {}
11        for col in global_original_columns:
12            if col == global_target_column:
13                continue
14            raw = request.form.get(f'input_{col}', '')
15            try:
16                val = float(raw)
17            except:
18                val = raw
19            input_data[col] = val
20
21        # Buat DataFrame input
22        input_df = pd.DataFrame([input_data])
23
24        # Gabungkan dengan training untuk jaga konsistensi one-hot encoding
25        train_features_df = global_raw_df.drop(columns=[global_target_column])
26        combined_raw = pd.concat([input_df, train_features_df], ignore_index=True, sort=False)
27        combined_encoded = pd.get_dummies(combined_raw, drop_first=True)
28
29        final_input = combined_encoded.iloc[0].reindex(
30            index=global_feature_names, fill_value=0
31        ).values.reshape(1, -1)
32
33        prediction = global_model.predict(final_input)[0]
34
35        return render_template('predict.html',
36                               columns=global_original_columns,
37                               target_column=global_target_column,
38                               prediction_result=str(prediction))
39
40    # GET -> tampilkan form kosong
41    return render_template('predict.html',
42                           columns=global_original_columns,
43                           target_column=global_target_column,
44                           prediction_result=None)

```

Penjelasan:

- Metode GET: Menampilkan form kosong di predict.html dengan field berdasarkan kolom original (kecuali target).
- Metode POST: Mengumpulkan input dari form, mengonversi ke float jika numerik.
- Preprocessing Input: Membuat DataFrame input, menggabungkan dengan data training raw untuk memastikan *one-hot encoding* konsisten (menggunakan `pd.concat` dan `pd.get_dummies`).
- Prediksi: Menyesuaikan input dengan fitur training menggunakan `reindex`, kemudian memprediksi dengan `global_model.predict`.
- Render Hasil: Mengirim prediksi kembali ke template untuk ditampilkan.

5. Main Block: Menjalankan Aplikasi

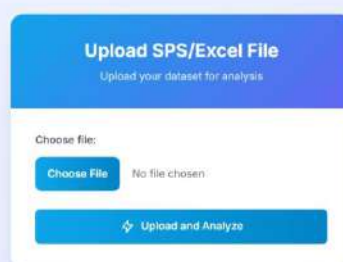
Cuplikan ini menjalankan server Flask dalam mode debug.

```
45
46 if __name__ == '__main__':
47     app.run(debug=True)
```

Penjelasan:

Menjalankan aplikasi dengan `debug=True` untuk logging error di console. Saat run, Flask otomatis melayani `/static` untuk file statis (CSS/JS/gambar di folder static). Untuk `/tmp`, bisa ditambahkan route khusus jika perlu menyimpan file sementara seperti plot tambahan.

Tampilan UI & Output



Decision Tree Analysis

Analysis results for your dataset

Target Column

Outcome

All Data (sample)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31.0	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1
8	125	96	0	0	0.0	0.232	54	1

Variables (X) and Class (y)

X (Variables)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
6	148	72	35	0	33.6	0.627	50
1	85	66	29	0	26.6	0.351	31
8	183	64	0	0	23.3	0.672	32
1	89	66	23	94	28.1	0.167	21
0	137	40	35	168	43.1	2.288	33
5	116	74	0	0	25.6	0.201	30
3	78	50	32	88	31.0	0.248	26
10	115	0	0	0	35.3	0.134	29
2	197	70	45	543	30.5	0.158	53
8	125	96	0	0	0.0	0.232	54

y (Class)

Outcome
1
0
1
0
1
0
1
0
1
1

Training / Testing Split (80-20)

X_train (sample)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
1	90	62	12	43	27.2	0.580	24
5	126	78	27	22	29.6	0.439	40
2	105	58	40	94	34.9	0.225	25
1	146	56	0	0	29.7	0.564	29
0	95	64	39	105	44.6	0.366	22
5	189	64	33	325	31.2	0.583	29
0	138	60	35	167	34.6	0.534	21
7	178	84	0	0	39.9	0.331	41
2	90	60	0	0	23.5	0.191	25
6	92	92	0	0	19.9	0.188	28

y_train (sample)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
1	90	62	12	43	27.2	0.580	24
5	126	78	27	22	29.6	0.439	40
2	105	58	40	94	34.9	0.225	25
1	146	56	0	0	29.7	0.564	29
0	95	64	39	105	44.6	0.366	22
5	189	64	33	325	31.2	0.583	29
0	138	60	35	167	34.6	0.534	21
7	178	84	0	0	39.9	0.331	41
2	90	60	0	0	23.5	0.191	25
6	92	92	0	0	19.9	0.188	28

X_test (sample)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
7	159	64	0	0	27.4	0.264	40
10	68	106	23	49	35.5	0.265	47
2	122	76	27	200	35.9	0.483	26
7	114	64	0	0	27.4	0.732	34
2	74	0	0	0	0.0	0.102	22
3	100	68	23	81	31.6	0.949	28
8	105	100	36	0	43.3	0.239	45
8	197	74	0	0	25.9	1.191	39
2	100	64	23	0	29.7	0.368	21
0	165	76	43	255	47.9	0.259	26

y_test (sample)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
7	159	64	0	0	27.4	0.294	40
10	68	106	23	49	35.5	0.285	47
2	122	76	27	200	35.9	0.483	26
7	114	64	0	0	27.4	0.732	34
2	74	0	0	0	0.0	0.102	22
3	100	68	23	81	31.6	0.949	28
8	105	100	36	0	43.3	0.239	45
8	197	74	0	0	25.9	1.191	39
2	100	64	23	0	29.7	0.368	21
0	165	76	43	255	47.9	0.259	26

y_pred (sample)

[illegible]

Accuracy

77.92%

Classification Report

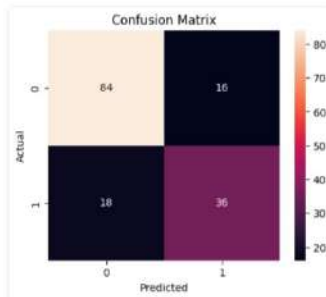
```

    "0": {
      "f1-score": 0.8336831683168316,
      "precision": 0.8235294117647058,
      "recall": 0.84,
      "support": 100.0
    },
    "1": {
      "f1-score": 0.6792452038188679,
      "precision": 0.6923076923076923,
      "recall": 0.6666666666666666,
      "support": 54.0
    },
    "accuracy": 0.7992207792207793,
    "macro avg": {
      "f1-score": 0.7556642256678498,
      "precision": 0.7579185206319193,
      "recall": 0.7533333333333333,
      "support": 154.0
    },
    "weighted avg": {
      "f1-score": 0.7782309228227405,
      "precision": 0.777516081460127,
      "recall": 0.7792207792207793,
      "support": 154.0
    }
  }
}

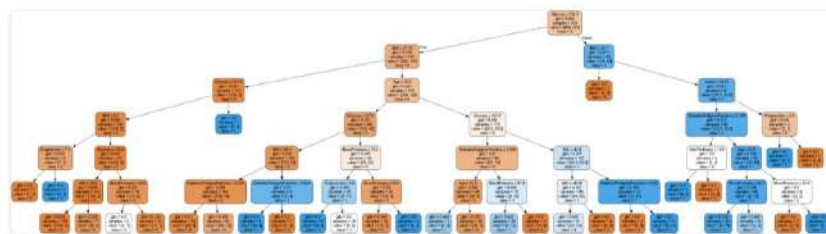
```

Confusion Matrix

[[84, 16], [18, 36]]



Decision Tree Visualization

[Go to Prediction Page](#)[Upload another dataset](#)

Predict New Input

Masukkan nilai fitur untuk memprediksi target class

Prediction Result

Negatif

Masukkan nilai untuk 'Pregnancies':	Masukkan nilai untuk 'Glucose':
2	100
Masukkan nilai untuk 'BloodPressure':	Masukkan nilai untuk 'SkinThickness':
72	20
Masukkan nilai untuk 'Insulin':	Masukkan nilai untuk 'BMI':
0	25.5
Masukkan nilai untuk 'DiabetesPedigreeFunction':	Masukkan nilai untuk 'Age':
12	50

⚡ Predict

Upload another dataset

Predict New Input

Masukkan nilai fitur untuk memprediksi target class

Prediction Result

Positif

Masukkan nilai untuk 'Pregnancies':	Masukkan nilai untuk 'Glucose':
6	148
Masukkan nilai untuk 'BloodPressure':	Masukkan nilai untuk 'SkinThickness':
72	35
Masukkan nilai untuk 'Insulin':	Masukkan nilai untuk 'BMI':
0	33.6
Masukkan nilai untuk 'DiabetesPedigreeFunction':	Masukkan nilai untuk 'Age':
0.627	50

⚡ Predict

Upload another dataset

Link GitHub

<https://github.com/shendyttria/Decision-Tree-Web>